Pricing Baku Homes from Scratch: A NumPy Decision Tree and Pegasos SVM on bina.az Listings

Farhad Budagov Murad Atlukhanov Panah Gulueyev Mikayil Hasanov

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We predict residential sale prices in Azerbaijan from a raw scrape of 100,775 bina.az listings. After de-duplicating repeat scrapes, removing the target-leaking columns `unit_price` and `total_price`, and applying fixed domain outlier rules, 57,473 listings with 92 features remain. We frame two tasks: (A) regression of log(price) and (B) a premium/standard tier split at the training-set median of 209,000 AZN. We implement a CART decision tree (Gini, entropy and MSE criteria with four stopping rules) and a soft-margin SVM trained with the Pegasos sub-gradient method, extended with random Fourier features, using NumPy only. On a held-out test set our tree reaches RMSE 0.232 on log(price) ($R^2 = 0.864$, MAPE 15.4%) and F1 0.921 on the tier task. The linear Pegasos SVM reaches F1 0.900 with ROC-AUC 0.965. Each result is within 0.001 of the matched scikit-learn baseline and within one cross-validation standard deviation. A random forest built from our own trees lowers the regression error to 0.185. The errors concentrate on the cheapest and most expensive listings, houses and villas, and sparsely listed micro-locations. Code: `github.com/pnah6699-maker/bina-az` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, regression, classification, real-estate price prediction.

I. Introduction

A listing price on bina.az, Azerbaijan's largest real-estate portal, is the seller's opening position in a negotiation. An estimate built from a home's size, location and condition helps buyers spot over-priced listings and helps sellers price realistically. The data is a genuine scrape, so most of the work is turning it into an honest modelling dataset. Our contributions:

- a documented cleaning pipeline that removes repeat scrapes, target leakage and data-entry errors, with every row count logged (Table 1);
- a from-scratch CART tree whose splits provably match scikit-learn on a fully determined toy example, sped up by pre-sorting each feature once;
- a from-scratch Pegasos SVM with margin and support-vector reporting, plus a random-Fourier-feature (RFF) non-linear version;
- a leakage-free protocol (validation-only tuning, refitting all preprocessing inside every CV fold) and a matched benchmark against scikit-learn, followed by an error analysis;
- all three bonuses: Ridge via the normal equations, a random forest of our own trees, and k-means/PCA geospatial analysis.

II. Background

Decision trees split a node with N samples on the feature/threshold pair that maximises the information gain $\mathrm{IG} = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$. Here I is the Gini impurity $1 - \sum_k p_k^2$, the entropy $-\sum_k p_k \log_2 p_k$, or, for regression, the node variance $\frac{1}{N}\sum_i (y_i - \bar{y})^2$ [2]. *Soft-margin SVMs* minimise the regularised hinge loss

$$J(\mathbf{w}, b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big), \quad (1)$$

which is equivalent to the usual C-formulation with $C = 1/(\lambda n)$. *Pegasos* [1] minimises (1) by stochastic sub-gradient steps with $\eta_t = 1/(\lambda t)$ and needs no QP or SMO solver. *Random Fourier features* [3] map $\mathbf{x}$ to $\mathbf{z}(\mathbf{x}) = \sqrt{2/D}\cos(W\mathbf{x} + \mathbf{c})$ with $W_{ij} \sim \mathcal{N}(0, 2\gamma)$, so that $\mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}') \approx e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$. A linear SVM on $\mathbf{z}$ then approximates an RBF-kernel SVM.

III. Data and Preprocessing

A. Dataset and cleaning

We use the Kaggle dataset `sehriyarmemmedli/binaaz-sale-project` (version 1, file `house_sale.csv`): 100,775 rows and 51 columns, scraped 5–16 October 2024. Every price is quoted in AZN; the code still maps currencies explicitly. Table 1 lists each cleaning step. (i) *Duplicates:* 36,321 rows repeat an item URL scraped on another day, so we keep the latest scrape per URL. A further 753 re-posts share price, area, rooms, floor, coordinates and category. (ii) *Scope:* we keep the three residential categories: new builds, old stock and houses/villas. Land is priced per *sot* (100 m^2), and commercial, office and garage listings form other markets. (iii) *Type coercion:* strings such as `"145 m²"`, `"4.5 sot"` and `"7 / 9"` become area in m^2, land area in m^2, floor and total floors. The district (*rayon*) and a metro tag are parsed from `extra_info`. (iv) *Outliers*, with fixed domain rules rather than data-driven percentiles, so that no statistic is computed on test rows: price in [10 k, 5 M] AZN, area in [15, 1000] m^2, a price per m^2 in [300, 15 000] used only to drop entry errors and never as a feature, and coordinates inside Azerbaijan. Rooms above 10 and inconsistent floors (floor $>$ total, or more than 40 floors) are set to missing rather than dropped. (v) *Missing values* (rooms 9%, floor for apartments, repair status 6%) are imputed with the training median, and a missing-indicator column is added for each. Houses have no floor, so their indicator is always 1.

Table 1. Cleaning log (rows left after each step).

Step	Rows left
raw rows	100,775
known currency, numeric price	100,775
unique listing URL (latest scrape)	64,454
residential categories	58,452
no exact re-posts (same price/area/rooms/floor/coords)	57,699
price in [10,000, 5,000,000] AZN	57,682
area in [15, 1000] m^2	57,594
price/m^2 in [300, 15,000] (entry errors)	57,474
coordinates inside Azerbaijan	57,473

Table 2. Dataset summary after cleaning.

Property	Value
Rows (after cleaning)	57,473
Features used	92
Train / Val / Test	40,233 / 8,620 / 8,620
Tier threshold (train median)	209,000 AZN
Premium share train/val/test (%)	49.7 / 49.8 / 49.6

B. Leakage

`unit_price` ("3 450 AZN/m²") equals price divided by area, and `total_price` equals `price` in all but 13 rows. Both are arithmetic functions of the target and are dropped before any other step (`LEAKAGE_COLUMNS` in `src/data_prep.py`). We also drop identifiers and personal data (`owner_name`, `shop_name`, raw `address`, URLs and image links), scrape metadata, free text, and `views`. View counts accumulate *after* a listing is posted and partly react to its price, so they would not be available at prediction time. A unit test (`test_leakage_columns_are_dropped`) checks that no feature name contains "price".

C. Features, tasks and split

The 92 features are: 13 numeric (area, log area, rooms, area per room, floor, total floors, floor ratio, first-floor and top-floor flags, land area, lat, lng, and haversine distance to the city centre); 9 binary (repaired, bill of sale, mortgage, agent, residential complex, VIP, featured, Baku, metro tag); 4 missing-value flags; and one-hot columns for category, 11 districts and 46 micro-locations. Districts and locations with fewer than 150 training rows are pooled into "other". We use one-hot encoding because it imposes no false order on districts and suits both trees and the SVM. Every learned quantity in the `Featurizer` (medians and category levels) is fit on the training split only.

Task A regresses log(price), since the raw price has skewness 5.5 against 0.29 after the log (Fig. 1a). *Task B* labels a listing *premium* if its price exceeds the *training* median of 209,000 AZN. The validation and test labels reuse that threshold. The 70/15/15 split is stratified on price deciles, so all three splits have the same price distribution and therefore the same tier balance (Table 2).

IV. Method

A. Decision tree (from scratch)

`DecisionTree(task, criterion, max_depth, min_samples_split, min_samples_leaf, min_impurity_decrease)` mirrors scikit-learn's API. *Split search:* each feature is sorted once at the root. A child inherits its rows in sorted order by stably filtering its parent's order, which costs $O(Nd)$ per level instead of $O(Nd \log N)$. For a node, all $N-1$ cut positions of all features are scored in one vectorised pass. Classification uses cumulative class counts, giving p_k on each side of every cut. Regression uses cumulative $\sum y$ and $\sum y^2$ on centred targets, giving the SSE on each side. A cut is valid only between two distinct values and only if both children keep at least `min_samples_leaf` rows. The threshold is the midpoint of the two neighbouring values, and a row goes left iff $x_j \le t$. *Stopping:* a node becomes a leaf at `max_depth`, when it has fewer than `min_samples_split` rows, when it is pure, or when the weighted decrease $\frac{N_t}{N}$ IG falls below `min_impurity_decrease` (scikit-learn's definition). A unit test shows that each of the four rules changes the number of leaves. *Leaves* predict the mean (regression) or the class frequencies, which give `predict_proba`, with the majority class as the label. *Importances* sum each feature's weighted impurity decrease over all splits; they match scikit-learn to 10^{-10} in a test. After fitting, the tree is flattened into arrays for vectorised prediction. *Ties:* we keep the first best split (lowest feature index, then lowest threshold). scikit-learn scans features in a random order, which explains most structural differences (§B).

B. Support-vector machine (from scratch)

`PegasosSVM` minimises (1) with labels mapped to ± 1. The bias is folded in as a constant feature, as the project statement suggests. Each epoch visits a fresh random permutation in mini-batches of $k = 32$. At step t, with A_t^+ the batch rows whose hinge is active ($y_i \mathbf{w}^\top \mathbf{x}_i < 1$), the update is

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda)\, \mathbf{w} + \tfrac{\eta_t}{k} \textstyle\sum_{i \in A_t^+} y_i \mathbf{x}_i, \tag{2}$$

followed by the optional Pegasos projection onto $\|\mathbf{w}\| \le 1/\sqrt{\lambda}$, which tames the very large early steps. The schedule is a hyper-parameter: $1/(\lambda t)$ (default), $\eta_0/\sqrt{t}$, or constant. J is evaluated on the full training set after every epoch. Features are standardised with training means and standard deviations, because the hinge loss is scale-sensitive. `margin_width` $= 2/\|\mathbf{w}\|$, and `support_vector_mask` returns the training rows with $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 1$. *Non-linear extension:* we chose **random Fourier features** ($D = 1000$) followed by the same linear solver. Kernelised Pegasos needs $O(n)$ kernel evaluations per step and stores up to n coefficients. With $n \approx 40$k that is impractical, whereas RFF keeps each step $O(D)$. A unit test confirms that $\mathbf{z}^\top \mathbf{z}$ approximates the RBF kernel to within 0.05, and that RFF-Pegasos solves XOR, which the linear model cannot.

C. Bonus models

Ridge solves $(X^\top X + \lambda I)\mathbf{w} = X^\top \mathbf{y}$ on centred data, leaving the intercept unpenalised. *Random forest*: 30 of our trees,

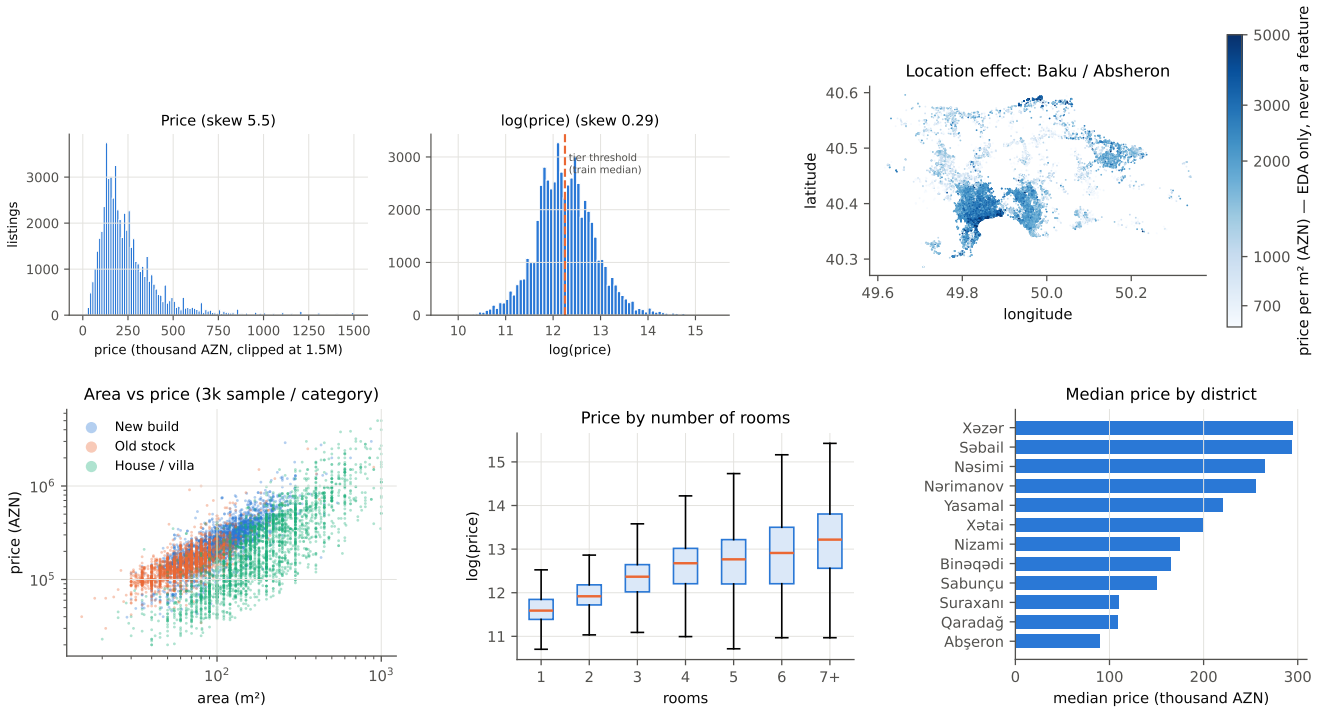


Figure 1. EDA on the 57,473 cleaned listings. (a) Price is heavily right-skewed and log(price) is nearly symmetric; the dashed line is the tier threshold. (b) Price per m^2 peaks in central Baku (Sabail, Nasimi, Yasamal). (c) Price scales roughly as a power of area, with houses priced lower per m^2. (d) Price grows monotonically with rooms. (e) The median price differs about 4× between Sabail and the outer districts.

each on a bootstrap sample with a random 33% of the features considered at every split; the forest averages predictions or probabilities.

V. Experimental Setup

Metrics: for Task A, RMSE, MAE and R^2 on log(price), plus MAPE and R^2 in AZN. For Task B, precision, recall and F1 for the premium class, ROC-AUC and PR-AUC (average precision), and the confusion matrix. All metrics are our own NumPy code and are tested against `sklearn.metrics`. *Protocol:* hyper-parameters are chosen on the validation split only. Final models are fit on the training split and scored *once* on the test split, with a bootstrap 95% CI (1000 resamples). For uncertainty we also run 5-fold CV on train+val, stratified on price deciles, re-fitting the featurizer, tier threshold and scaler inside every fold. *Search grids:* trees use `max_depth` $\in \{2,\dots,25\} \times$ `min_samples_leaf` $\in \{1,5,20,50\}$ (Task A) and $\{1,20\} \times$ {Gini, entropy} (Task B). The SVM uses $\lambda \in [10^{-5}, 10^{-1}]$ with 30 epochs; when validation F1 is within 0.002, we take the larger λ, i.e. the wider margin. RFF uses $\gamma \in [10^{-3}, 10^{-1}] \times \lambda \in \{10^{-3}, 10^{-4}\}$. *Baselines with matched settings:* `DecisionTreeRegressor` and `DecisionTreeClassifier` with the same depth, leaf size and criterion; `SGDClassifier`(hinge, $\alpha = \lambda$, same epochs); `LinearSVC`($C = 1/(\lambda n)$); `RBFSampler`+SGD with the same γ, D and λ; an RBF `SVC` on a 10k-row subsample, since exact SVC is quadratic in n; and `Ridge` and `RandomForestRegressor` with the same settings. Every scikit-learn SVM is wrapped in a `Pipeline` with a `StandardScaler`. Trivial mean and majority baselines anchor the scales. All randomness uses seed 42. `python -m src.run_all` regenerates every number, table and figure in 11 minutes on a laptop CPU.

VI. Results

A. Tuning (validation only)

Tree, Task A (Fig. 2a). With `min_samples_leaf` $= 1$, training RMSE falls steadily with depth while validation RMSE bottoms out at depth 10–12 (0.240) and then rises: this is the variance side of the bias–variance trade-off. Raising the leaf size to 5 or 20 removes the rise and lowers validation RMSE to 0.226. We select depth 17 with leaf size 5. The learning curve (Fig. 2c) shows validation RMSE still falling from 0.308 with 2k rows to 0.226 with 40k rows, with a persistent train/validation gap, so more data would still help. *Tree, Task B* (Fig. 2b). Gini beats entropy at every depth up to 12. Both reach about 0.925 validation F1 around depth 14–17 and fit the training set perfectly by depth 25. Leaf size 20 costs about 0.008 F1 here. We select Gini, depth 14, leaf size 1.

SVM (Table 3, Fig. 2d, Fig. 3). Increasing λ from 10^{-5} to 10^{-1} shrinks $\|\mathbf{w}\|$ from 18.9 to 1.13. The margin therefore widens 17-fold, and the share of support vectors rises from 16% to 41%. Validation F1 is flat at 0.906–0.909 for $\lambda \in [3 \cdot 10^{-4}, 3 \cdot 10^{-2}]$ and drops at both ends: over-regularised at large λ, and not converged in 30 epochs at $\lambda = 10^{-5}$, where $J = 0.80$. We select $\lambda = 10^{-2}$. Its objective falls from $J = 1.00$ at $\mathbf{w} = \mathbf{0}$ to 0.270 after one epoch and 0.256 after 30. All three learning-rate schedules converge to the same $J \approx 0.256$, but the constant step is noisier. Running more epochs widens the margin slightly, from 0.895 after one epoch to 0.960 after 40. The 2-D slice in Fig. 3 makes the effect of λ visible: $\lambda = 1$ gives a margin band of width 3.8 that contains

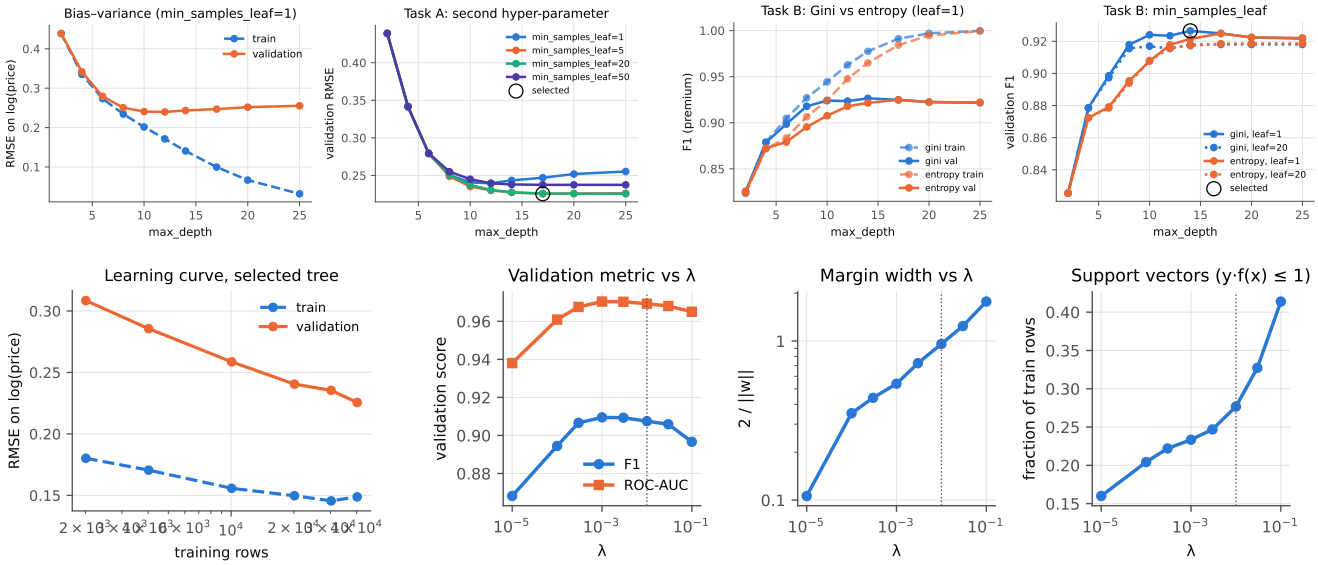


Figure 2. Validation studies. (a) Task A: depth sweep with train vs. validation RMSE, and a second sweep over `min_samples_leaf`. (b) Task B: Gini vs. entropy across depths and leaf sizes. (c) Learning curve of the selected tree. (d) SVM: as λ grows, the margin $2/\|\mathbf{w}\|$ widens and more points become support vectors; validation F1 and AUC peak at $\lambda \approx 10^{-3}$–10^{-2} (dotted line: selected).

Table 3. Pegasos λ sweep (validation, 30 epochs).

λ	val F1	val AUC	$\|\mathbf{w}\|$	$2/\|\mathbf{w}\|$	SV %
0.1	0.897	0.965	1.13	1.771	41.4
0.03	0.906	0.968	1.61	1.242	32.7
0.01	0.908	0.969	2.09	0.959	27.7
0.003	0.909	0.970	2.76	0.725	24.7
0.001	0.909	0.970	3.71	0.539	23.3
0.0003	0.907	0.968	4.55	0.440	22.2
0.0001	0.894	0.961	5.68	0.352	20.4
1e-05	0.868	0.938	18.85	0.106	16.0

Table 4. Headline results: test value (5-fold CV mean ± std).

Model	Task A RMSE (log)	Task B F1
Ours: decision tree	0.232 (0.232±0.004)	0.921 (0.926±0.004)
Ours: SVM (Pegasos)	n/a	0.900 (0.907±0.003)
Ours: SVM + RFF	n/a	0.900 (0.906±0.003)
sk: decision tree	0.232 (0.233±0.004)	0.921 (0.926±0.004)
sk: SGD hinge	n/a	0.900 (0.908±0.003)

most points. For RFF, the best setting is a wide kernel ($\gamma = 0.003$, $\lambda = 10^{-4}$); validation F1 falls steadily for $\gamma \geq 0.03$.

B. *Test-set comparison*

Table 4 summarises the headline results, and Tables 5–6 list every model. *Task A:* our tree reaches RMSE 0.232 on log(price) (95% bootstrap CI $[0.225, 0.240]$), $R^2 = 0.864$ and MAPE 15.4%. In AZN this is an MAE of 44.6 k and $R^2 = 0.763$, about 2.7 times better than predicting the mean. *Task B:* our tree reaches F1 0.921 (CI $[0.915, 0.928]$). The linear SVM reaches F1 0.900 (CI $[0.893, 0.907]$) but a much higher ROC-AUC (0.965 vs. 0.917), because the tree's scores are coarse leaf frequencies while the SVM produces a continuous ranking. RFF matches but does not beat the linear SVM. Once price, area and distance are on log or standardised scales, the premium/standard boundary is close to linear, and the trees' advantage comes from interactions that a wide RBF kernel smooths away.

Ours vs. scikit-learn. On a fully determined 8-row example, both our classification trees (Gini and entropy) and our regression tree reproduce scikit-learn's split features and thresholds exactly (`tests/test_decision_tree.py`). On five random continuous regression problems, the predictions agree to 10^{-10}. On the real data (Table 7) the trees have the same depth and leaf counts within 4, and agree on 94.5% (A) and 98.9% (B) of test predictions. Every metric matches to within 0.002. The differences come from ties. In Task A the root split is `area_m2` ≤ 100.1 for us and `log_area` ≤ 4.606 for scikit-learn. Since $e^{4.606} = 100.1$, both produce exactly the same partition with the same gain: we keep the lower feature index, while scikit-learn's random feature order picked the other. Classification gains are ratios of small integers, so exact ties are even more common deeper in the tree. Thresholds also differ in the last float bits (90.15 vs. scikit-learn's float32 90.149998). Pegasos and `SGDClassifier` agree to the third decimal on every metric, and `LinearSVC`, which solves the dual exactly, is only 0.0025 F1 higher. Our Ridge coefficients equal scikit-learn's to $8 \cdot 10^{-11}$. *Speed:* our tree fits in 6.6 s (A) and 4.3 s (B), against scikit-learn's compiled Cython at 0.5 s, i.e. 9–12× slower. Pegasos takes 2.1 s against 0.4 s for SGD. Our 30-tree forest takes 132 s against 4.4 s, but is equally accurate (RMSE 0.185; Fig. 4g). Prediction is under 0.01 s for every model except exact SVC, which needs 4.9 s even when trained on only 10k rows; that cost is why we did not use it as a main model.

VII. Error Analysis and Discussion

What the models use. Area dominates the tree importances (0.60 for A, 0.54 for B), followed by distance to the centre (0.22, 0.15) and raw lat/lng (Fig. 5). The largest SVM

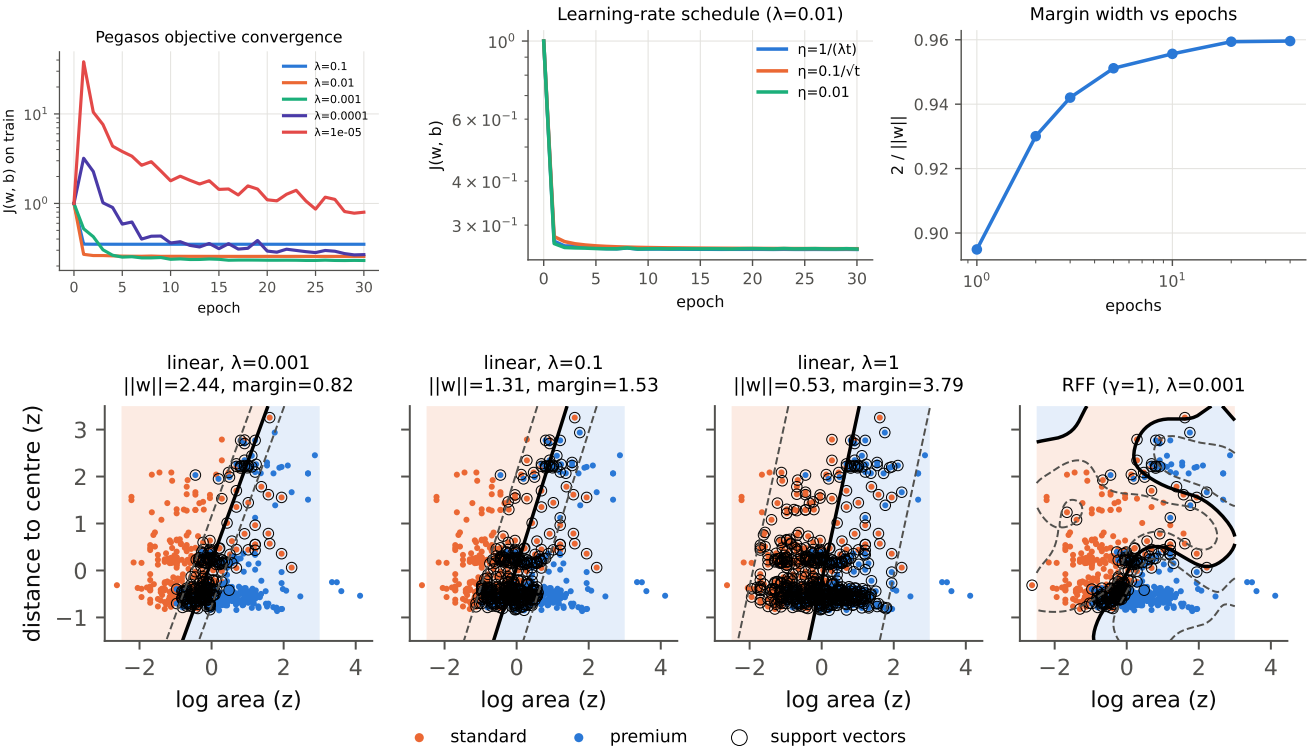


Figure 3. Pegasos diagnostics. Top: the training objective J per epoch for several λ; the effect of the learning-rate schedule; and margin width vs. epochs. Bottom: a 2-D illustration on two standardised features (log area, distance to centre; 600 training rows). Solid line: decision boundary; dashed lines: $f(\mathbf{x}) = \pm 1$; circled points: support vectors. Larger λ gives a wider margin and more support vectors. The RFF panel shows a curved boundary.

weights tell a consistent story: $+1.49$ for log area, $+0.55$ for rooms, and $+0.40$ for the premium Ag Şeher ("White City") development; -0.31 for the Absheron district, -0.27 for houses/villas, and -0.27 for distance to the centre.

Luxury and bargain outliers. The tree shrinks predictions toward the middle (Fig. 4a). It over-predicts the cheapest decile by $+0.11$ in log terms (12%) and under-predicts the most expensive decile by -0.12. MAPE is about 12% in the middle deciles but 26% and 22% at the two ends. Because leaves average, the most extreme prices cannot be reached. The worst errors (`results/error_worst.csv`) are implausibly cheap listings, e.g. a 120 m^2 new build near the 20 Yanvar metro listed at 47 k AZN. These are almost certainly down payments or monthly instalments quoted as the price, which our price/m^2 floor of 300 does not catch.

Houses and sparse districts. Houses/villas have 32% MAPE against 12% for apartments. Their value depends on land, build quality and plot shape, and the scrape describes these poorly. Micro-locations with fewer than 150 training listings have 29.9% MAPE against 14.3% for well-covered ones, and test MAPE correlates with log(train count) at $r = -0.53$ (Fig. 4c). The k-means clusters (bonus, Fig. 6) show the same pattern geographically: RMSE is 0.19–0.21 in the dense central and eastern Baku clusters (9–25k training rows) and 0.34–0.41 on the northern Absheron coast. In clusters outside Absheron with only 2–26 training rows, RMSE reaches 0.85–0.89.

Tier errors sit at the boundary. For listings within 5% of the threshold price, 38% (tree) and 46% (SVM) of predictions are wrong. Beyond 50% from the threshold, the error rates are 1.9% and 1.4% (Fig. 4d). Most Task-B errors are therefore unavoidable label noise: two near-identical flats a few thousand AZN apart fall on opposite sides of an arbitrary median.

Limitations. (1) These are *asking* prices from an 11-day snapshot; transaction prices and seasonal effects are not observed. (2) Stratifying the split on price deciles uses each row's own price to assign it to a split. This never enters a feature, a threshold or a model, but strict deployment would use a time-based split. (3) The fixed outlier rules remove some genuine luxury listings above 5 M AZN, and keep some mis-entered instalment prices. (4) The free-text description, which mentions renovations, documents and views, is unused. (5) The forests and the exact SVC were evaluated only on the test split, not with CV. (6) Our NumPy tree is about $10\times$ slower than compiled code, which limited how many forest configurations we could tune.

VIII. Conclusion

A carefully cleaned and leakage-free version of the bina.az scrape lets simple from-scratch models predict Baku sale prices to within about 15% MAPE and classify the price tier with F1 0.92. Our NumPy tree and Pegasos SVM match scikit-learn to within 0.002 on every metric; the remaining differences come from tie-breaking and float precision, not from modelling. The decision tree is the stronger classifier at a fixed threshold, while the SVM is the better ranker. Averaging our own trees in a forest gives the largest single gain

Table 5. Task A, regression on log(price), test set. CV = 5-fold mean ± std on train+val (the forests were not cross-validated, to save time). [b]bonus. Fit time on one CPU core.

Model	RMSE	MAE	R^2	MAPE%	CV RMSE	fit s
Mean baseline	0.629	0.487	0.000	55.2	0.630±0.003	0.00
Ours: decision tree	0.232	0.148	0.864	15.4	0.232±0.004	6.60
sk: DecisionTreeRegressor	0.232	0.148	0.864	15.4	0.233±0.004	0.53
Ours: Ridge[b]	0.242	0.168	0.852	17.1	0.240±0.004	0.03
sk: Ridge	0.242	0.168	0.852	17.1	0.240±0.004	0.08
Ours: random forest[b]	0.185	0.115	0.914	11.7	—	131.76
sk: RandomForestRegressor	0.185	0.116	0.914	11.8	—	4.36

Table 6. Task B, premium vs. standard tier, test set (positive class = premium). The majority baseline predicts "standard" for every row.

Model	Prec.	Rec.	F1	AUC	CV F1	fit s
Majority baseline	0.000	0.000	0.000	0.500	0.000±0.000	0.00
Ours: decision tree	0.918	0.925	0.921	0.917	0.926±0.004	4.28
sk: DecisionTreeClassifier	0.919	0.923	0.921	0.916	0.926±0.004	0.47
Ours: Pegasos SVM (linear)	0.894	0.906	0.900	0.965	0.907±0.003	2.06
sk: SGDClassifier hinge	0.893	0.907	0.900	0.965	0.908±0.003	0.37
sk: LinearSVC	0.895	0.910	0.902	0.968	0.909±0.003	0.73
Ours: RFF Pegasos SVM	0.883	0.918	0.900	0.967	0.906±0.003	5.44
sk: RBFSampler + SGD hinge	0.886	0.915	0.901	0.967	0.907±0.003	2.23
sk: SVC rbf (10k sub.)	0.898	0.901	0.899	0.967	—	1.93

Table 7. Tree structure: ours vs. scikit-learn (same hyperparameters).

Task	leaves	depth	root split (ours / sklearn)	same pred.%
A	3945 / 3944	17 / 17	area_m2≤100.1 / log_area≤4.606	94.5
B	1521 / 1525	14 / 14	area_m2≤90.15 / area_m2≤90.15	98.9

(RMSE 0.232 → 0.185). Next we would add text features from the descriptions, use a time-based split, target-encode the sparse micro-locations, and try gradient boosting with our trees.

Tools and Acknowledgements

AI-assistant disclosure. We used Claude Code (Anthropic), an AI coding assistant, to draft the code in `src/` and `tests/`, the plotting scripts, and a first version of this report's text. Every member reviewed, ran and edited the parts listed against their name in `contribution_report.md`, and all reported numbers come from running `python -m src.run_all`. **Data:** bina.az sale listings, Kaggle dataset `sehriyarmemmedli/binaaz-sale-project`; we do not redistribute the raw data or personal fields. **Software:** NumPy, pandas and Matplotlib; scikit-learn was used only for baselines, bonus clustering/PCA, and sanity checks in tests.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[3] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems (NIPS)*, 2007, pp. 1177–1184.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

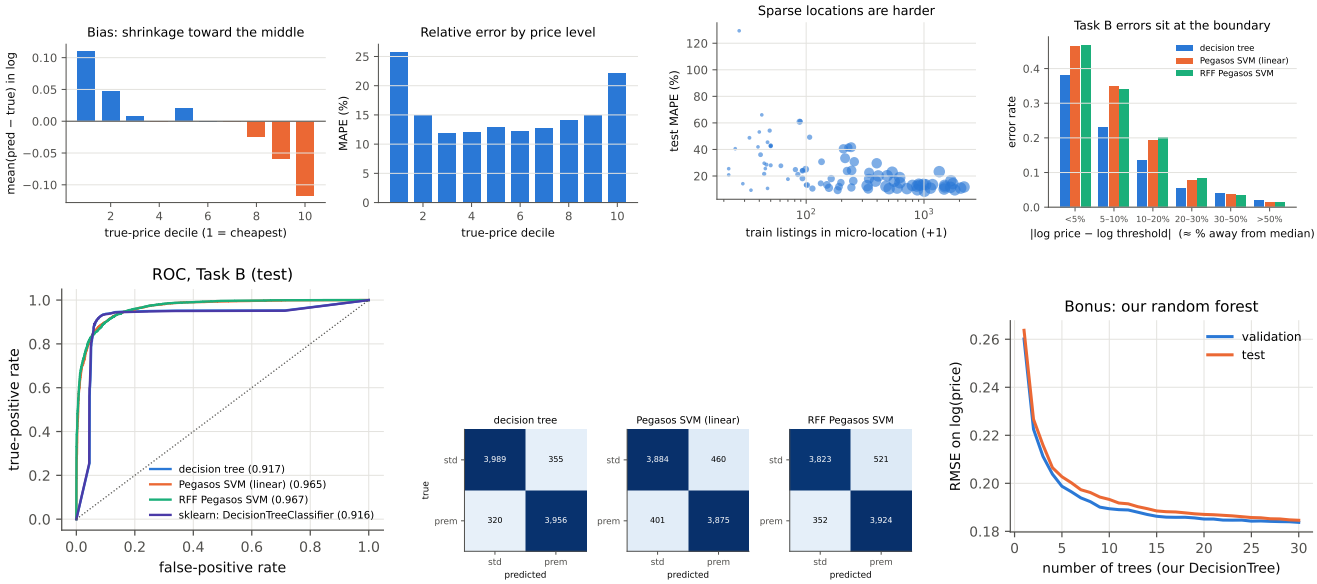


Figure 4. Error analysis on the test set. (a,b) The tree shrinks extreme prices toward the middle; relative error is highest at both ends. (c) Rarely listed micro-locations have higher error. (d) Tier misclassifications concentrate near the threshold. (e) ROC curves: the SVM ranks better (higher AUC) even though the tree has the higher F1. (f) Confusion matrices. (g) Bonus: the RMSE of our random forest falls from 0.264 with one tree to 0.185 with 30 trees.

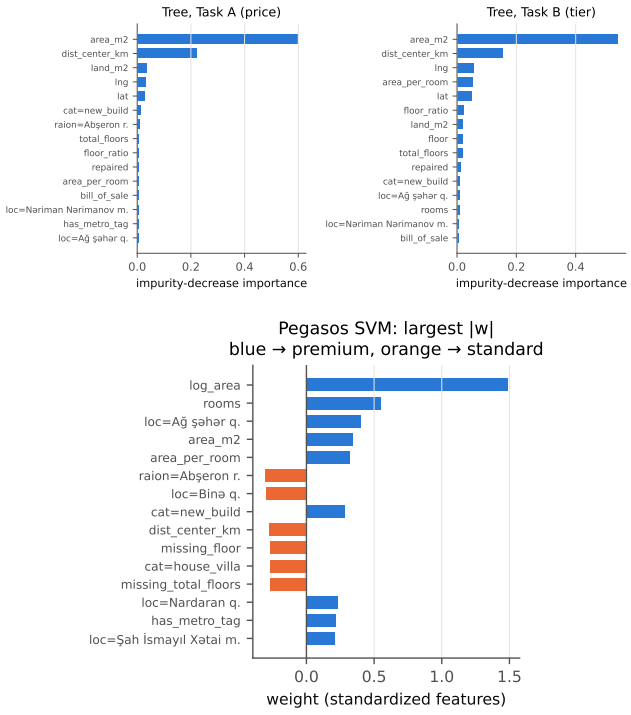


Figure 5. Impurity-decrease importances of our trees (top) and the largest Pegasos weights on standardised features (bottom).

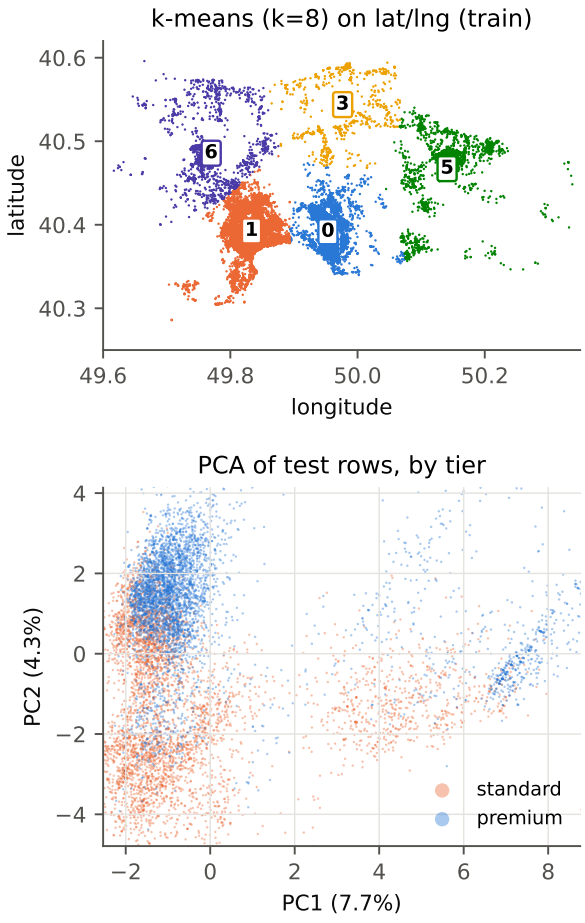


Figure 6. Bonus: k-means ($k = 8$) on training coordinates, used to break down errors by area. A PCA projection of the test rows shows that the tiers overlap heavily in the first two components (7.7% and 4.3% of the variance), so no single direction separates them.